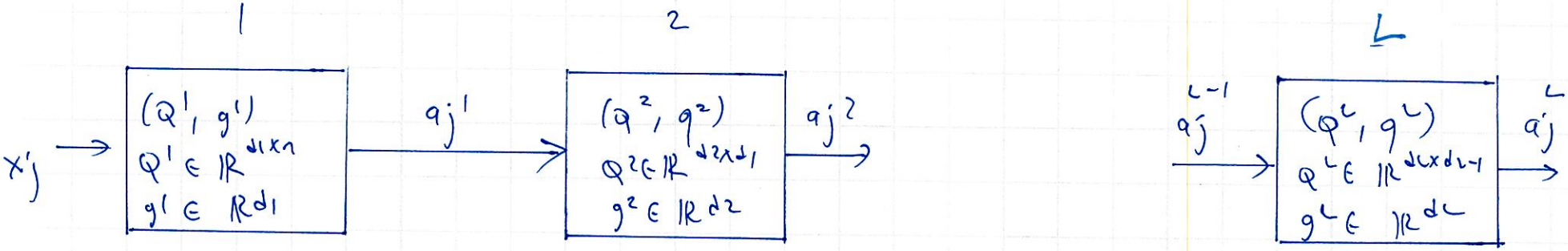
Deep learning

Consider a multi-class classification problem with data points $\{(x_j, y_j) : j = 1, \dots, m\}$ where $x_j \in \mathbb{R}^n$ and $y_j \in \{1, 2, \dots, K\}$



$$a_j^1 = \sigma_1(\underbrace{\underbrace{Q^1}_{d_1 \times n} \underbrace{x_j}_{n \times 1} + g^1}_{d_1 \times 1}) \qquad a_j^2 = \sigma_2(\underbrace{\underbrace{Q^2}_{d_2 \times d_1} \underbrace{a_j^1}_{d_1 \times 1} + g^2}_{d_2 \times 1}) \quad \dots \quad a_j^L = \sigma_L(\underbrace{\underbrace{Q^L}_{d_L \times d_{L-1}} \underbrace{a_j^{L-1}}_{d_{L-1} \times 1} + g^L}_{d_L \times 1})$$

Each element of $\sigma_1, \sigma_2, \dots \sigma_L$ could be

$\sigma(t) = 1/(1+e^{-t})$

$\sigma(t) = \max\{t, 0\}$

If we use a neural net with parameters $(Q^1, g^1, Q^2, g^2, \dots Q^L, g^L)$, then corresponding to the input x_j, the output a_j^L is given by the function:

$$Z(x_j, Q^1, g^1, Q^2, g^2, \dots, Q^L, g^L) =$$

$$\underbrace{\sigma_L\Big(Q^L \underbrace{\sigma_{L-1}\Big(\cdots \underbrace{\sigma_2\Big(Q^2 \underbrace{\sigma_1\big(Q^1 x_j + g^1\big)}_{\text{output from layer 1}} + g^2\Big)}_{\text{output from layer 2}} \cdots\Big)}_{\text{output from layer } L-1} + g^L\Big)}_{\text{output from layer } L}$$

Solve a multi-class classification problem by using $Z(x_j, Q^1, g^1, \dots, Q^L, g^L)$ as the feature vector for data point j.

data point #	feature vector	label	likelihood
1	$Z(x_1, Q^1, g^1, \dots, Q^L, g^L) = Z_1$	2	$e^{w_2^t Z_1} \Big/ \sum_{l=1}^{K} e^{w_l^t \cdot Z_1}$
2	$Z(x_2, Q^1, g^1, \dots, Q^L, g^L) = Z_2$	4	$e^{w_4^t Z_2} \Big/ \sum_{l=1}^{K} e^{w_l^t \cdot Z_2}$
3	$Z(x_3, Q^1, g^1, \dots, Q^L, g^L) = Z_3$	1	$e^{w_1^t Z_3} \Big/ \sum_{l=1}^{K} e^{w_l^t Z_3}$
$\vdots$		$\vdots$	
M	$Z(x_M, Q^1, g^1, \dots, Q^L, g^L) = Z_M$	4	$e^{w_4^t \cdot Z_M} \Big/ \sum_{l=1}^{K} e^{w_l^t Z_M}$

Say $(w_1, w_2, \dots, w_K)$ are the parameters for each class, where $w_1, w_2, \dots, w_K \in \mathbb{R}^{d_L}$

Likelihood of the whole dataset

$$L(Q^1, g^1, \dots, Q^L, g^L, w_1, w_2, \dots, w_K)$$

$$= \frac{e^{w_2^t \cdot Z(x_1, Q^1, g^1, \dots, Q^L, g^L)}}{\sum_{l=1}^{K} e^{w_l^t Z(x_1, Q^1, g^1, \dots, Q^L, g^L)}} \cdots \frac{e^{w_4^t Z(x_M, Q^1, g^1, \dots, Q^L, g^L)}}{\sum_{l=1}^{K} e^{w_l^t Z(x_M, Q^1, g^1, \dots, Q^L, g^L)}}$$

$$\ln L(Q^1, g^1, \dots, Q^L, g^L, w_1, w_2, \dots, w_K)$$

$$= -\sum_{j=1}^{M} \ln\left(\sum_{\ell=1}^{K} e^{w_\ell^t Z(x_j, Q^1, g^1, \dots, Q^L, g^L)}\right)$$

$$+ \sum_{j=1}^{M} \sum_{k=1}^{K} \mathbb{1}(y_j = k) \; w_k^t \cdot Z(x_j, Q^1, g^1, \dots, Q^M, g^M)$$

The maximum likelihood problem we solve is:

$$\max_{\substack{(Q^1, g^1) \in \mathbb{R}^{d_1 \times (n+1)} \\ (Q^2, g^2) \in \mathbb{R}^{d_2 \times (d_1+1)} \\ \vdots \\ (Q^L, g^L) \in \mathbb{R}^{d_L \times (d_{L-1}+1)} \\ (w_1, w_2, \dots, w_K) \in \mathbb{R}^{K \times d_L}}} \left\{ -\sum_{j=1}^{M} \ln\left(\sum_{\ell=1}^{K} e^{w_\ell^t Z(x_j, Q^1, g^1, \dots, Q^L, g^L)}\right) + \sum_{j=1}^{M} \sum_{k=1}^{K} \mathbb{1}(y_j = k) \; w_k^t \cdot Z(x_j, Q^1, g^1, \dots, Q^M, g^M) \right\}$$

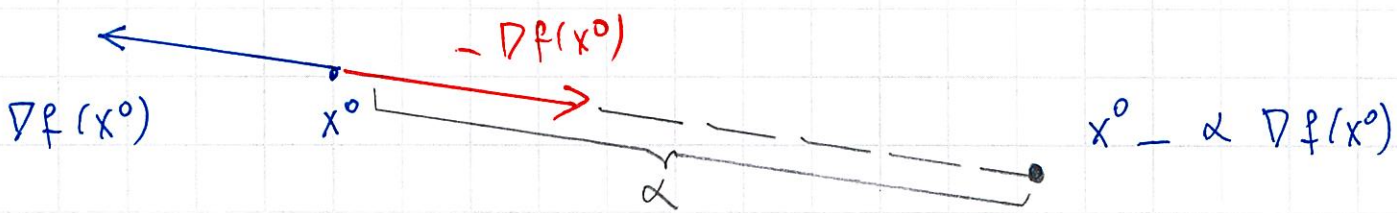
Basics of Gradient Descent Methods

Let $f: \mathbb{R}^n \to \mathbb{R}$ be a function that maps n-dimensional vectors to a scalar. The gradient of f at point x^0 is the n-dimensional vector given by

$$\nabla f(x^0) = \left(\frac{\partial f(x)}{\partial x_1}\bigg|_{x=x^0}, \frac{\partial f(x)}{\partial x_2}\bigg|_{x=x^0}, \dots, \frac{\partial f(x)}{\partial x_n}\bigg|_{x=x^0} \right)$$

- The general principle is that if we start at point x^0 and move in the direction of the gradient $\nabla f(x^0)$ for a small step size, the value of the function gets larger

- If we start at some point x^0 and move against the direction of the gradient $\nabla f(x^0)$ for a small step size, the value of the function gets smaller



If α is small enough then

$$f(x^0 - \alpha \nabla f(x^0)) \leq f(x^0)$$

Example: Consider the function: $f(x) = x_1^2 + 4x_1x_2 + 3x_2^2 - 5$

$$\nabla f(x) = \begin{pmatrix} 2x_1 + 4x_2 \\ 4x_1 + 6x_2 \end{pmatrix}$$

- Let's start at point $x^0 = (1,1)^t$ and move against the gradient for a step size of 0.1.

$$\nabla f((1,1)^t) = \begin{pmatrix} 6 \\ 10 \end{pmatrix}$$

The new point we reach is $\begin{pmatrix} 1 \\ 1 \end{pmatrix} - 0.1 \cdot \begin{pmatrix} 6 \\ 10 \end{pmatrix} = \begin{pmatrix} 0.4 \\ 0 \end{pmatrix}$

$f(x^0) = f((1,1)^t) = 3$ $\quad$ $f(x^0 - \alpha \nabla f(x^0)) = f((0.4, 0)^t) = -4.84$

Indeed $\quad f(x^0) = 3 \geq -4.84 = f(x^0 - \alpha \nabla f(x^0))$

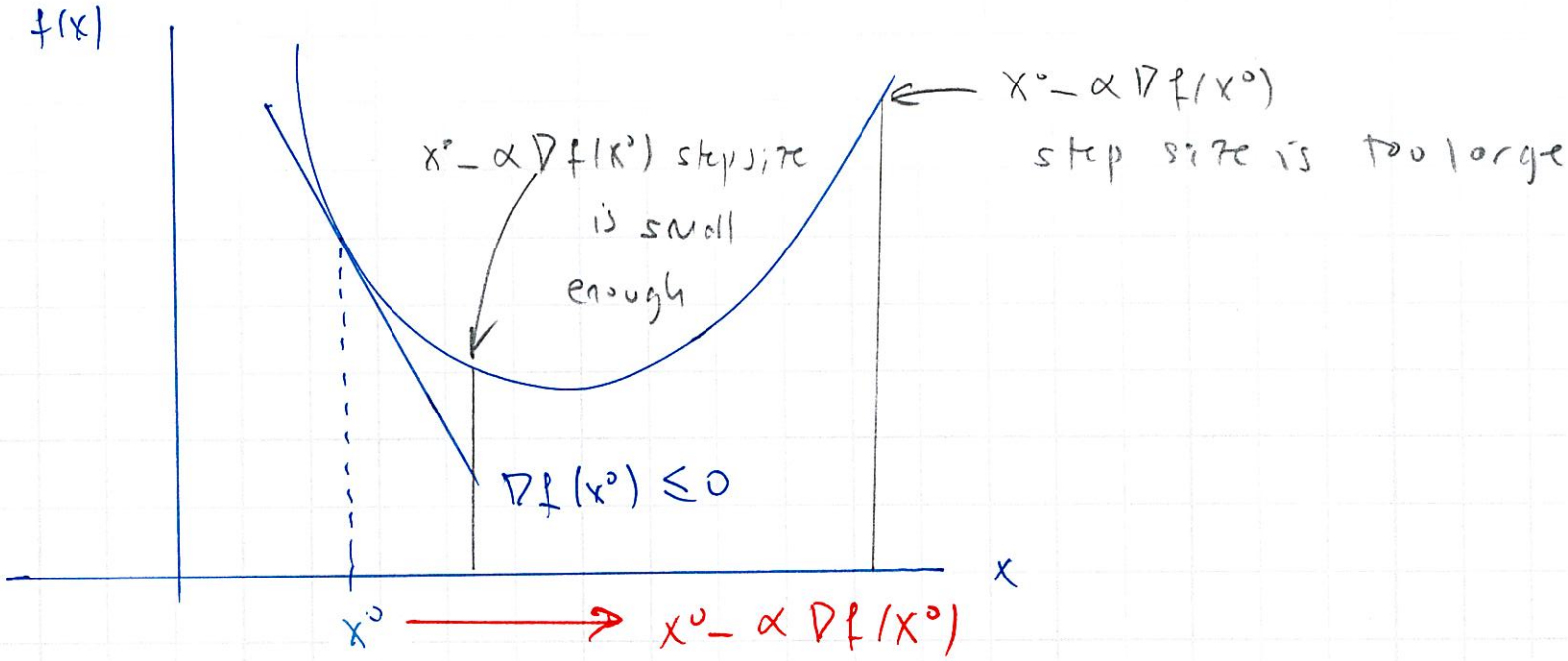
- Let's start at the point $x^0 = (1,1)^t$ and move against the gradient for a step size of 0,5

$$\nabla f((1,1)^t) = \begin{pmatrix} 6 \\ 10 \end{pmatrix}$$

The new point we reach is $\begin{pmatrix} 1 \\ 1 \end{pmatrix} - 0{,}5 \cdot \begin{pmatrix} 6 \\ 10 \end{pmatrix} = \begin{pmatrix} -2 \\ -4 \end{pmatrix}$

$f(x^0) = f((1,1)^t) = 3$ $\quad$ $f(x^0 - \alpha \nabla f(x^0)) = f((-2,-4)^t) = 79$

We do not have $f(x^0) = 3 \geqslant 79 = f(x^0 - \alpha \cdot \nabla f(x^0))$



Because $\nabla f(x^0) \leq 0$, $x^0 - \alpha \nabla f(x^0) \geq x^0$

- We immediately get a crude algorithm to minimize a function. Start at any point x^0 and generate a sequence of points $x^1, x^2, x^3, \ldots$ by moving against the gradient for small step sizes.

- At iteration k of the algorithm, we are at point x^k. Say we use the step size α^k. Then the next point we reach is

$$x^{k+1} = x^k - \alpha^k \nabla f(x^k)$$

- Find α^k by "trial and error". In particular, pick any value for α^k. Check if $f(x^k - \alpha^k \nabla f(x^k)) < f(x^k)$

if yes, take the step
if no, halve α^k and try again.